Problem 3: Row With Maximum Ones(easy)

Problem Statement

Given a binary matrix that has dimensions $m * n$, consisting of ones and zeros, determine the row that contains the highest number of ones and return two values: the zero-based index of this row and the actual count of ones it possesses.

If there is a tie, i.e., multiple rows contain the same maximum number of ones, we must select the row with the lowest index.

Examples

Example 1:

- **Input**: `[[1, 0], [1, 1], [0, 1]]`
- **Expected Output**: `[1, 2]`
- **Justification**: The second row `[1, 1]` contains the most ones, so the output is `[1, 2]`.

Example 2:

- **Input**: `[[0, 1, 1], [0, 1, 1], [1, 1, 1]]`
- **Expected Output**: `[2, 3]`
- **Justification**: The third row `[1, 1, 1]` has the most ones, leading to the output `[2, 3]`.

Example 3:

- **Input**: `[[1, 0, 1], [0, 0, 1], [1, 1, 0]]`
- **Expected Output**: `[0, 2]`
- **Justification**: Both the first and third rows contain two ones, but we choose the first due to its lower index, resulting in `[0, 2]`.

Try it yourself

Try solving this question here:

Java

```java
public class Solution {
    public int[] findMaxOnesRow(int[][] mat) {
        int maxOnesIdx = 0;
        int maxOnesCount = 0;
        // ToDo: Write Your Code Here.
        return new int[]{maxOnesIdx, maxOnesCount};
    }
}
```

Test Case | Run Code Result | Stdout

```
[[1,0],[0,1],[1,1]]
```

Use Example Testcases | Run | Submit

← Previous

Solution: Matrix Diagonal Sum (Easy)

Next →

Solution: Row With Maximum Ones(Easy)

Mark as Completed